A Reference Architecture for Sovereign AI in School Divisions

Design, Integration, and Privacy Considerations for On-Premises AI Assistants in K–12 Educational Institutions

Gord Tulloch
Director, Information and Communication Technology
Sunrise School Division
gtulloch@sunrisesd.ca

June 2026

Abstract

The deployment of artificial intelligence (AI) assistants in K–12 school Divisions raises profound questions about student data privacy, institutional sovereignty, and technological governance. Cloud-hosted AI services, while convenient, require transmitting sensitive student records beyond the institution's security boundary — a practice incompatible with FIPPA, PHIA, and Divisional Policies. This paper presents a reference architecture for a fully sovereign, on-premises AI assistant for school Division staff, realized through the Soleil Suite: a production system comprising four tightly integrated components — SoleilESB (Enterprise Service Bus), SoleilDW (Data Warehouse), SoleilMCP (Model Context Protocol Server), and SoleilAI (Conversational AI Client). The architecture demonstrates that educational institutions can deliver natural-language access to authoritative student data — attendance, enrollment, rosters, performance, and transportation — without transmitting student records to external AI providers. Key design principles include broker-mediated data access, defense-in-depth row-level security, the Model Context Protocol as a standardized tool interface, retrieval-augmented generation grounded in institution-controlled documents, and Microsoft Entra ID federated identity propagated end-to-end without credential substitution. The paper describes the architecture of each component, the data flows that connect them, the security invariants enforced at every layer, and the implementation decisions that distinguish this approach from cloud-dependent alternatives.

Keywords: *sovereign AI, K–12 education, student data privacy, FIPPA, PHIA, Model Context Protocol, retrieval-augmented generation, enterprise service bus, data warehouse, large language models, on-premises AI, federated identity, row-level security.*

1. Introduction

School Divisions generate and steward some of the most sensitive personal data in the public sector: student academic records, attendance histories, behavioral incidents, transportation assignments, and family contact information. The emergence of large language model (LLM) technology has created genuine demand from teachers, principals, and administrators for natural-language access to this data — the ability to ask, in plain English, "Which of my students have been absent more than three times this month?" and receive an accurate, role-scoped answer drawn from authoritative records.

The dominant commercial AI offerings satisfy this demand by routing queries and, often, underlying data to vendor-operated cloud infrastructure. For school Divisions this is categorically unacceptable: student records protected under Canada's PIPEDA, and provincial privacy acts such as FIPPA and PHIA cannot be disclosed to third parties without explicit consent — consent that a blanket SaaS agreement does not provide. Beyond legal compliance, institutional leaders rightfully question whether an AI assistant's responses should be grounded in vendor-curated training data rather than in the Division's own authoritative records.

This paper describes a reference architecture — instantiated as the Soleil Suite at Sunrise School Division — that addresses both concerns. The suite delivers a fully sovereign AI assistant: the LLM runs on Division-owned hardware; student data never leaves the institutional network; and every response is grounded in records drawn directly from the Division's own data warehouse through a multi-layer security model that enforces per-user, per-role data scoping at the warehouse level, not as an afterthought in application code.

The architecture is organized around four components whose responsibilities are cleanly separated and whose integration points are precisely specified:

- **SoleilESB** — an Enterprise Service Bus that extracts data from operational systems (such as PowerSchool, Routefinder, etc.) and publishes it to a shared RabbitMQ message broker.
- **SoleilDW** — a Data Warehouse that ingests, transforms, and dimensionalizes staged data into a star-schema model with a rigid security and access-control domain, exposed externally only through broker-mediated named queries over semantic views.
- **SoleilMCP** — a Model Context Protocol server that acts as the AI data gateway, translating LLM tool calls into authenticated DataRequest messages to SoleilDW and managing a security-classified document corpus for retrieval-augmented generation (RAG).
- **SoleilAI** — a conversational AI client (browser + FastAPI backend) that authenticates staff via Microsoft Entra ID, orchestrates the LLM tool-call loop, injects RAG context, and streams responses token-by-token to the browser.

The paper proceeds as follows: Section 2 surveys the problem context and motivations for sovereign AI in educational settings. Sections 3 through 6 describe each component in depth. Section 7 presents the end-to-end data flow for a complete AI query. Section 8 analyzes the security architecture. Section 9 discusses implementation considerations and lessons learned. Section 10 concludes with implications for the broader K–12 sector.

2. Problem Context and Motivation

2.1 The Privacy Imperative

Student data occupies a unique position in privacy law. Privacy legislation grants parents and eligible students rights over educational records and restricts their disclosure without consent. Cloud AI services typically process user inputs on vendor infrastructure, creating disclosure events that implicate these statutes even when the service agreement includes data processing addenda. The legal and reputational exposure of a disclosure incident involving student records creates a compelling institutional motive for keeping AI inference entirely within the Division's own network boundary.

2.2 The Grounding Problem

Beyond privacy, cloud AI assistants face a fundamental epistemological problem in educational contexts: they cannot know the Division's data. A teacher asking about attendance trends for their specific class cannot receive an accurate answer from a model trained on generic internet text. The answer requires access to attendance records for that teacher's enrolled students, drawn from the Division's Student Information System (SIS). Without a direct, secure connection to authoritative data, the AI assistant can only hallucinate plausible-sounding but factually unsupported responses — an outcome worse than no response at all in a professional context where accuracy matters.

Retrieval-Augmented Generation (RAG) and tool-call integration address this problem architecturally: by equipping the LLM with the ability to query live data sources and retrieve relevant document context, responses can be grounded in institutional facts rather than statistical approximations. The challenge is implementing these integrations in a way that preserves the privacy boundary.

2.3 The Role Scoping Problem

Educational data access is inherently hierarchical and role-differentiated. A teacher should see data only for their own enrolled students. A principal should see data for all students at their school. A Superintendent should have Division-wide access. An administrative staff member's scope depends on their specific operational assignment. Any AI assistant integrated with live student data must enforce these distinctions rigorously — not merely as a user-interface convention but as an architectural invariant that cannot be circumvented by clever prompt construction. It should also log what is accessed.

The Soleil architecture addresses this through a defense-in-depth model with two independent enforcement points: per-tool permission checks in SoleilMCP and precomputed row-level security in SoleilDW, each operating independently without knowledge of or reliance on the other's controls.

2.4 Defining Sovereignty

For the purposes of this paper, "sovereign AI" denotes an AI system in which: (a) LLM inference executes on infrastructure owned or exclusively leased by the institution; (b) student records and other sensitive data are not transmitted beyond the institution's network perimeter during query processing; (c) the institution retains full control over the model selection, system prompting, tool availability, and access-control policy; and (d) the system is auditable — every data access can be traced to a specific user, query, and timestamp without reliance on a third party's audit infrastructure.

3. SoleilESB: The Enterprise Service Bus

3.1 Role in the Architecture

SoleilESB is the data acquisition layer of the Soleil Suite. It is responsible for extracting records from operational source systems — principally PowerSchool (the Student Information System) and Routefinder (the transportation management system) — and publishing them as JSON messages to a shared RabbitMQ message broker. SoleilESB operates as an intermediary that normalizes and decouples the operational systems from the analytical infrastructure downstream.

3.2 Connector Architecture

SoleilESB implements a template-driven connector framework. Each connector is a Python module implementing one of five patterns, organized into producers (which extract and publish data) and consumers (which transform and forward it):

Producer Patterns:

- **APIExtractor** — extracts records from REST API endpoints (e.g., the PowerSchool API) and publishes each record as a JSON message to a named RabbitMQ exchange.
- **DBExtractor** — executes a SQL query or stored procedure against a database and publishes the result set as a stream of JSON messages.

Consumer Patterns:

- **SFTPForwarder** — consumes messages from a source exchange and delivers them to external applications via SFTP, applying configurable field mapping and filtering.
- **Aggregator** — combines output from multiple producer exchanges into a single consolidated output (e.g., for producing composite student records).
- **Slicer** — consumes large messages and splits them into smaller units, resubmitting individual records to downstream exchanges.

3.3 Template-Driven Connector Development

A key productivity feature of SoleilESB is its web-based connector scaffolding system. Administrators navigate a guided form — selecting a system, a pattern, a connection, and pattern-specific parameters — and the system automatically generates a working Python connector file by merging the selections into a template. Variable substitution covers connector name, connection credentials, source and target exchanges, SQL queries, and SFTP paths. The generated file is immediately registered with the monitoring system and available for manual review and customization before production deployment.

This approach significantly reduces the barrier to creating new data integrations: a staff member with modest Python familiarity can produce a functional, tested connector within minutes of defining the data source and destination.

3.4 Connectors for the AI Pipeline

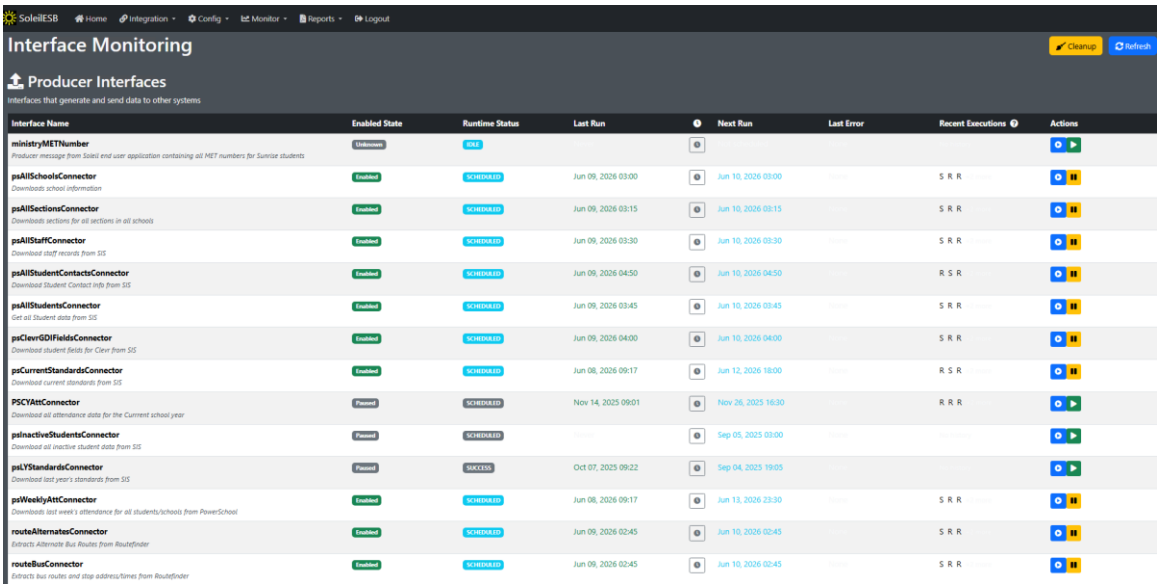
The connectors relevant to the AI pipeline extract the primary educational datasets from PowerSchool and Routefinder and publish them to dedicated RabbitMQ exchanges:

Connector	Source System	Exchange	Data Content
psAllStudentsConnector	PowerSchool API	ps.allstudents	Student master records (demographics, enrollment)
psAllSectionsConnector	PowerSchool API	ps.allsections	Course/section definitions with teacher assignments
psAllStaffConnector	PowerSchool API	ps.allstaff	Staff records with roles and school assignments
Connector	**Source System**	**Exchange**	**Data Content**
PSCYAttConnector	PowerSchool API	ps.attendance	Daily attendance events (present/absent/tardy)
routeBusConnector	Routefinder API	routefinder.buses	Bus route definitions and school assignments
routeAlternatesConnector	Routefinder API	routefinder.alternates	Alternate route configurations

Table 1: SoleilESB connectors feeding the AI data pipeline.

3.5 Scheduling and Monitoring

SoleilESB runs as a Django application backed by Celery and Django Celery Beat for distributed task scheduling. Each connector is registered as a Celery task with a configurable schedule. The Celery Flower dashboard provides real-time visibility into task execution, queue depths, and worker health. A built-in Interface Monitoring view tracks the last execution time and message count for every connector, enabling operators to detect stale data flows without manual log inspection.



4. SoleilDW: The Data Warehouse

4.1 Role in the Architecture

SoleilDW is the analytical backbone of the Soleil Suite. It receives raw message streams from SoleilESB via RabbitMQ, stages them in auto-provisioned PostgreSQL tables, and promotes them through a pluggable ETL framework into a dimensionalized star-schema warehouse. It exposes data externally only through a broker-mediated named query interface — no external HTTP API, no direct database access for any consumer. This design makes SoleilDW the sole enforcement point for data governance: every query that reaches the LLM has passed through SoleilDW's named query registry, consumer permission model, and row-level security filters.

4.2 Data Ingestion and Staging

A long-running Django management command (*IngestData*) subscribes to all configured RabbitMQ exchanges and writes incoming JSON messages to PostgreSQL staging tables. A distinctive architectural feature is *automatic staging table provisioning*: when the first message arrives on a new exchange, IngestData executes a CREATE TABLE statement naming the table after the exchange. Subsequent messages that contain previously unseen fields trigger ALTER TABLE ADD COLUMN operations. This zero-touch onboarding means that adding a new data source requires only registering the exchange — no manual DDL, no Django migrations, no application restart.

Schema evolution is strictly additive: columns are added but never automatically dropped or renamed, preserving data integrity across source system changes. Every DDL event is recorded in a StagingSchemaEvolutionLog for auditability.

4.3 ETL Framework and Dimensional Model

A pluggable ETL task framework promotes staged data into the dimensional model. Each ETL task is a Python class implementing a BaseETLTask interface with extract(), transform(), and load() methods. Tasks are registered in a database table and scheduled via APScheduler; adding a new task requires only creating a Python module and a registry row — no framework changes. The dimensional model follows a conformed star schema:

Table	Type	Purpose
DimStudent	Type-2 SCD	Student demographics, enrollment status, parent contacts. History-preserving.
DimSchool	Type-1	School master records with region and type.
DimStaff	Type-1	Staff records with role and primary school.
DimSection	Type-1	Course/section definitions with teacher and school references.
DimRoute	Type-1	Bus route definitions.
DimDate	Type-1	Standard calendar dimension.
FactAttendance	Fact	Per-student, per-date attendance events with codes and minutes absent.
FactSectionEnrollment	Fact (security-critical)	Student–section–teacher assignments driving row-level security.
FactTransportation	Fact	Student–route assignments.
FactStandardsPerformance	Fact	Student achievement scores by curriculum standard.

Table 2: SoleilDW dimensional model — dimensions and fact tables.

DimStudent is implemented as a Type-2 slowly-changing dimension: when a tracked attribute changes, the prior row is expired (EffectiveEndDate set, IsCurrent = false) and a new current row inserted. This preserves historical snapshots — essential for answering questions about a student's status as of a past date. Each current DimStudent row additionally carries a *StudentProfileText* column: a generated natural-language summary of demographic and enrollment attributes, pre-computed for use by the RAG subsystem in SoleilMCP.

4.4 Security and Access Control as a Data Domain

The most architecturally distinctive aspect of SoleilDW is its treatment of security as a first-class data domain rather than as an application-layer filter. The design principle — *"security is not a filter, it is a data domain"* — is carried through into a dedicated set of warehouse tables:

- **DimUser**: maps each Entra ID User Principal Name to a surrogate UserKey, linking to DimStaff where applicable.
- **UserSectionAccess**: explicit grants of a user to a section (teacher's own classes).
- **UserSchoolAccess**: explicit grants of a user to a school (principal's own school).
- **UserGlobalAccess**: explicit global-scope grants (superintendent, system admin).
- **UserStudentAccessResolved**: a precomputed table mapping each UserKey to the set of StudentKeys that user is authorized to see.

UserStudentAccessResolved is rebuilt by a scheduled AccessResolutionETL job by joining FactSectionEnrollment with the grant tables. This precomputation strategy eliminates complex runtime joins on every query: any student-scoped named query can filter its result using a single, indexed predicate — WHERE StudentKey IN (SELECT StudentKey FROM UserStudentAccessResolved WHERE UserKey = :user) — rather than re-resolving access from first principles at query time.

4.5 Semantic View Layer

Raw dimensional tables are never exposed to external consumers. Instead, SoleilDW defines a semantic view layer — PostgreSQL views that flatten joins, pre-aggregate common metrics, and retain the key columns needed for access-scope filtering. These views are the only objects ever referenced by named queries:

Semantic View	Purpose
vwStudent360	Flattened student demographics, school, enrollment, and transportation.
vwAttendanceSummary	Pre-aggregated AbsencesLast30Days, AbsencesYTD, LateCount per student. Backs get_student_attendan
vwAttendanceTrends	Attendance aggregated by day/week/month for trend analysis across classes, schools, and the Division.
vwClassRoster	Teacher–section–student enrollment list with grade context. Backs get_class_roster.
vwEnrollmentSummary	Enrollment counts and recent changes by grade, school, and Division.
vwPerformanceByStandard	Student achievement scores by curriculum standard. Also used by get_at_risk_students.
vwStudentTransportation	Bus route and stop assignments per student, plus transportation exception events.

Table 3: SoleilDW semantic views — the sole externally-queryable surface.

4.6 Broker-Mediated External Access

A second long-running management command, *ProcessRequests*, subscribes to a dedicated RabbitMQ exchange (soleildw.requests) and handles two message types:

- **DataRequest**: a JSON message containing a consumer_id, api_key, dataset (named query identifier), parameters (including the calling user's UPN/UserKey for RLS filtering), and a reply_to queue name. ProcessRequests authenticates the consumer, verifies per-dataset permission grants, executes the named query with bound parameters, and publishes the JSON result to the reply_to queue.
- **CatalogRequest**: a JSON message requesting the list of named queries the consumer is permitted to invoke. The reply catalog is filtered to the consumer's granted, published queries — consumers cannot discover the existence of datasets they are not permitted to access.

Every DataRequest and CatalogRequest execution is logged in DataRequestLog with consumer identity, dataset name, parameter names (never values), row count, duration, and outcome. No external system holds database credentials; all data access flows through this broker-mediated, authenticated, audited interface.

5. SoleilMCP: The Model Context Protocol Server

5.1 Role in the Architecture

SoleilMCP is the AI data gateway — the component that translates LLM-issued tool calls into authenticated, access-scoped data requests to SoleilDW. It also manages the retrieval-augmented generation (RAG) document corpus that grounds LLM responses in institutional documents. SoleilMCP implements the Model Context Protocol (MCP), an open standard for connecting LLMs to tools and data sources, developed by Anthropic and adopted broadly across the AI ecosystem.

Critically, SoleilMCP is not a trusted internal component in relation to SoleilDW. It authenticates with SoleilDW's Consumer Registry using an assigned consumer_id and api_key, identically to any other external consumer. SoleilDW does not grant SoleilMCP — or any consumer — elevated or unscoped access to the dimensional model.

5.2 Tool Architecture: DW Tools and Custom Tools

SoleilMCP exposes two categories of MCP tools to the LLM:

DW Tools are dynamically derived from SoleilDW's named query catalog. At startup, SoleilMCP sends a CatalogRequest to SoleilDW and upserts the returned named queries into its own Tool table. Each named query maps 1:1 to one MCP tool. Because the tool list is dynamically populated, adding a new named query to SoleilDW's catalog requires zero code changes in SoleilMCP — only a catalog sync. The tool's description field, authored in SoleilDW, includes representative natural-language prompt examples that enable the LLM to reliably select the correct tool for a given user question.

Custom Tools are hand-coded Python modules managed through SoleilMCP's administration UI. Each module must implement a standard interface: a TOOL_DEFINITION dict and an async execute(params, context) function. The admin UI includes a CodeMirror 6 code editor, save-as-draft / activate workflow, and an AI-assisted implementation feature backed by the internal Codex LLM service. Custom tools support any logic — calling external APIs, composing data from multiple sources — and share the same authentication and permission infrastructure as DW tools.

5.3 Authentication, Authorization, and Identity Propagation

Every MCP tool call carries an Entra ID Bearer token in the HTTP Authorization header. SoleilMCP validates the token using the MSAL library on every request. Before executing any tool, the dispatcher checks the ToolPermission table to verify that the authenticated principal (identified by their Entra Object ID or group membership) holds an explicit grant for the requested tool.

For DW tools, SoleilMCP resolves the caller's User Principal Name (UPN) from the validated token and injects it as a bound parameter on the DataRequest message published to SoleilDW. SoleilDW's named query templates use this UPN to look up the caller's UserKey and apply UserStudentAccessResolved-based row filtering before returning any data. SoleilMCP never substitutes its own consumer_id for the calling user's identity in this parameter — doing so would defeat SoleilDW's row-level security.

This design creates two independent access-control enforcement points: SoleilMCP's

ToolPermission check (which tools a user may invoke) and SoleilDW's ConsumerPermission + NamedQuery RLS (which rows a user may see within a tool's results). Neither layer alone is sufficient; both are required, implementing a defense-in-depth model.

The named query catalog published to SoleilMCP is configured to support the Division's full range of operational staff roles:

Role (Entra Group)	Permitted Tools (Named Queries)
Teachers	get_student_attendance_summary, get_attendance_trends, get_at_risk_students, get_class_roster, get_performance_summary
Principals	All Teacher tools + get_enrollment_summary, get_enrollment_changes, get_class_absentee_rates, get_student_performance_summary
Superintendent / Leadership	All Principal tools + get_school_absentee_comparison, get_Division_performance_summary
Administrative Staff	get_student_contact_info, get_student_details, get_enrollment_changes, get_student_transportation, get_student_performance_summary

Table 4: Role-to-tool permission configuration in SoleilMCP.

5.4 RAG Document Corpus and Secure Retrieval

SoleilMCP also hosts a RAG subsystem that enables the LLM to ground responses in institutional documents — policy guides, curriculum resources, operational procedures, board reports — without those documents needing to be fed into the LLM's context window permanently or sent to external embedding services.

Documents are managed through the SoleilMCP administration UI: an administrator uploads a PDF, DOCX, plain-text, or Markdown file and assigns it a security class from the following enumeration:

Security Class	Content Type	Permitted Roles
PUBLIC	Division-wide public information, calendars, announcements	Any authenticated user
TEACHER	Instructional and professional content, curriculum	Teachers, Principals, Superintendents, Admin Staffs, staff bulletins
PRINCIPAL	School-administration content, operational policies, principal circulars	Principals, Superintendents
SUPERINTENDENT	Division-leadership content, board reports, strategic plans	Superintendent / Leadership only
ADMIN_STAFF	Enrollment procedures, transportation protocols, administrative guides	Administrative Staff, Principals, Superintendents

Table 5: RAG document security classification model.

At ingest time, a background task extracts text, splits it into overlapping chunks (configurable size and overlap), embeds each chunk via an OpenAI-compatible embedding API (the internal Codex service at codex.sunrisesd.ca), and stores each chunk with its embedding vector in a pgvector column in SoleilMCP's PostgreSQL database. A separate HNSW index is maintained per security class, enabling pre-filtered approximate nearest-neighbor queries.

At query time, SoleilAI calls SoleilMCP's POST /tools/secure_retrieve endpoint. The endpoint validates the Bearer token, resolves the caller's permitted security classes from their Entra group memberships, embeds the query, and executes a cosine-similarity search with the security class filter applied as a WHERE predicate *inside* the pgvector query — never as a post-hoc filter on an unfiltered result set. This pre-filtering invariant ensures that restricted chunk vectors are never scored against

a user's query, eliminating the possibility of side-channel information disclosure through similarity scores.

The query string itself is never stored in any log or database record. Only its character count is recorded in the RAGQueryLog, preventing PII that users may include in natural-language questions from being persisted.

5.5 Audit Logging

Every MCP tool invocation produces an immutable AuditLog entry recording: timestamp, caller UPN, caller Entra OID, tool name, response status, duration in milliseconds, and IP address. Input parameters are *not* stored to avoid persisting student PII in audit records. AuditLog records are retained for a minimum of two years and cannot be deleted or modified through the administration UI.

6. SoleilAI: The Conversational AI Client

6.1 Role in the Architecture

SoleilAI is the user-facing component — the conversational interface through which teachers, principals, and administrators interact with the AI assistant. It is a Python/FastAPI backend serving a vanilla-JavaScript browser single page application (SPA), deployed as a Docker container behind the Soleil Suite's shared nginx reverse proxy at the /ai/ path prefix.

SoleilAI is strictly read-only: it does not modify any back-end system. It holds no duplicate access-control state; all permission enforcement and data scoping are delegated entirely to SoleilMCP and SoleilDW. Adding a new tool in SoleilMCP requires zero code changes in SoleilAI — the tool list is fetched dynamically from SoleilMCP at startup.

6.2 Authentication Flow

SoleilAI uses two Microsoft Entra ID application registrations:

- A **SPA registration** for the browser-side MSAL.js PKCE (Proof Key for Code Exchange) authorization-code flow. MSAL.js is served as a static asset — not loaded from a CDN — to eliminate external network dependency during school network outages.
- An **API registration** that defines the API scope. The FastAPI backend validates tokens issued for this audience using the python-jose library, verifying the RS256 signature against the Entra JWKS endpoint on every request, with a 1-hour key cache.

The validated Bearer token is forwarded verbatim to SoleilMCP for both MCP tool calls and RAG retrieval. SoleilAI never substitutes a service-account credential or re-issued token for the live per-user Bearer token — doing so would bypass SoleilMCP's per-tool permission checks and SoleilDW's row-level security. This token propagation invariant is the architectural mechanism by which the end-to-end identity chain is maintained from the browser through to the data warehouse.

6.3 The Tool-Call Orchestration Loop

When a user submits a chat message, the Tool Orchestrator executes a multi-iteration agentic loop:

1. Load conversation history from the SQLite store.
2. Call RAGClient to retrieve relevant document chunks from SoleilMCP's /tools/secure_retrieveendpoint (2-second timeout; failure is non-fatal).
3. Inject RAG chunks into the system message for this call (not persisted to conversation history).
4. Call the vLLM server with the message history, the live tool list from SoleilMCP, and theRAG-augmented system message.
5. If the LLM emits a function-call (tool invocation request):
6. Emit a tool_start SSE event to the browser.
7. Dispatch the call to MCPClient.call_tool() with the user's Bearer token.
8. If the result exceeds 10,000 characters, write it to a download file and emit a tool_download eventwith a trimmed preview.
9. Otherwise emit a tool_result event with the full result.

10. Inject the result into message history and loop back to step 4 (up to 10 iterations).
11. When the LLM emits a final text response, stream it as SSE token events.
12. Persist the completed assistant message to the Conversation Store and emit a done event.

6.4 Duplicate Tool-Call Guard

A notable safety feature is the duplicate tool-call guard: if the LLM attempts to call the same tool more than once within a single user turn, the second and subsequent calls are blocked. A tool_error SSE event is emitted, and all tools are stripped from the next LLM call, forcing the model to synthesize an answer from the existing results. This prevents runaway agentic loops in which a confused model repeatedly queries the same tool without making progress toward an answer.

6.5 LLM Backend

SoleilAI connects to a remote vLLM server via its OpenAI-compatible POST /v1/chat/completions endpoint. The vLLM server runs on Division-owned GPU hardware within the institutional network. The LLM model is configured at the infrastructure level — SoleilAI interacts with it exclusively through the OpenAI-compatible API, making the system model-agnostic. The vLLM server is not reachable from outside the Docker internal network; SoleilAI communicates with it via private Docker networking.

The LLM client implements control-token sanitization to handle models that leak chat-template tokens (e.g., <|im_start|>, <|im_end|>) into their output. Context-window management drops the oldest non-system messages when the estimated token count approaches the configured budget, preserving the system message and most recent turns.

6.6 Conversation Persistence and Privacy

Per-user conversation history is stored in a SQLite database mounted on a named Docker volume that survives container rebuilds. Conversations are keyed by the user's Entra UPN; a user can never access another user's conversations regardless of the supplied conversation_id.

Student data returned by tool calls exists only in the in-memory llm_messages list during a single chat turn. It is never persisted to the conversation database. This design means that if the conversation database were ever extracted, it would contain only user query text and LLM reasoning — not student records.

6.7 Browser SPA Features

The browser SPA is implemented in vanilla JavaScript without a build step, minimizing supply-chain risk and deployment complexity. It provides:

- Incremental token streaming rendered as GitHub-Flavored Markdown with syntax highlighting.
- Collapsible tool-call bubbles showing the tool name, formatted arguments, and result.
- A conversation sidebar listing the user's saved sessions, with rename and delete.
- Per-conversation settings: temperature, max tokens, context size, hide-tool-bubbles toggle.
- Optional voice input via the browser Web Speech API — client-side only, no audio transmitted tothe server.
- WCAG 2.1 Level AA accessibility compliance.

7. End-to-End Data Flow

7.1 Data Acquisition Flow

The data pipeline originates in SoleilESB. Scheduled Celery tasks invoke the configured APIExtractor or DBExtractor connectors, which call the PowerSchool or Routefinder APIs and publish each record as a JSON message to a named RabbitMQ exchange. SoleilDW's IngestData command consumes these messages, auto-provisions staging tables as needed, and writes each record. Scheduled ETL tasks promote staged records into the dimensional model. The AccessResolutionETL job rebuilds UserStudentAccessResolved from the updated
FactSectionEnrollment and access-grant tables on a schedule aligned with the MCP's access-scope cache TTL (proposed default: every 15 minutes).

7.2 Catalog Synchronization

At startup (and on demand), SoleilMCP sends a CatalogRequest to SoleilDW's soleildw.requests exchange. ProcessRequests authenticates the SoleilMCP consumer, filters the named query catalog to the consumer's granted, published queries, and publishes the catalog JSON to the reply_to queue. SoleilMCP upserts the returned named queries as Tool records and registers them with the MCP server. From this point, the LLM's tool list is live and accurate without any code deployment.

7.3 AI Query Flow (Complete)

A complete AI query — from a teacher's natural-language question to a data-grounded response — traverses the following path:

Step	Component	Action
1	Browser SPA	User authenticates via Entra PKCE. MSAL.js acquires a Bearer token scoped to the SoleilAI API registration.
2	Browser SPA	User submits: "Which of my students have been absent more than 3 times this month?"
3	SoleilAI FastAPI	EntraTokenValidator validates the Bearer token (RS256, issuer, audience, expiry).
4	SoleilAI / RAGClient	POST /tools/secure_retrieve → segmented RAG vector databases based on authorization
5	SoleilAI / Tool Orchestr	RAG chunks injected into system message. LLM called with message history + live tool list.
6	vLLM Server	LLM selects get_student_attendance_summary tool and emits a function-call with date_fro threshold
7	SoleilAI / MCPClient	Dispatches tool call to SoleilMCP with Bearer token.
8	SoleilMCP / Auth Engi	Token validated. ToolPermission checked: teacher role has access to get_student_attenda UPN
9	SoleilMCP / RabbitMQ	DataRequest published: { consumer_id, api_key, dataset: 'get_student_attendance_summa up
10	SoleilDW / ProcessRe	Consumer authenticated. ConsumerPermission verified. Named query executed against vwAttendanceSumma
11	SoleilDW	JSON result (only the teacher's students, only those with 3+ absences) published to reply_ queue.
12	SoleilMCP	Result returned to SoleilAI MCPClient. AuditLog entry written.
13	SoleilAI / Tool Orchestrator	Result event emitted to browser. Result injected into LLM message history.
14	vLLM Server	LLM synthesizes a natural-language response from the result data.
15	Browser SPA	Response streamed token-by-token via SSE. Rendered as Markdown unless the user specifically requests another format.

Table 6: End-to-end query flow for a teacher attendance question

8. Security Architecture

8.1 Defense-in-Depth Model

The Soleil architecture implements security through multiple independent layers, each enforcing its own access controls without relying on the correctness of adjacent layers. A failure in any single layer does not compromise the overall system's security posture.

Layer	Component	Control	What It Enforces
1 — Transport	nginx gateway	TLS 1.2+	All traffic encrypted in transit.
2 — Authentication	SoleilAI	Entra ID Bearer token validation (RS256, JWKS)	Only authenticated Division staff may submit queries.
3 — Tool permissions	SoleilMCP	ToolPermission table (Entra OID	Each role may invoke only the tools assigned to their group
4 — Consumer auth	SoleilDW	Consumer Registry (consumer_id + hashed api_key)	Only registered systems may send DataRequest messages.
5 — Dataset grants	SoleilDW	ConsumerPermission table	SoleilMCP may query only the named queri explicitly granted.
6 — Row-level security	SoleilDW	UserStudentAccessResolv	ed (precomputed, pre-filtered)Each user's q contain only rows they are authorized to see
7 — Network isolation	Docker networking	PostgreSQL port never published	No external system can reach the warehous database directly.
8 — RAG filtering	SoleilMCP	pgvector WHERE security class IN (...)	Only document chunks the user is authorized to see are returned.

Table 7: Defense-in-depth security layers.

8.2 Identity Propagation Without Credential Substitution

A critical security invariant of the architecture is that the calling user's Entra Bearer token is forwarded verbatim from SoleilAI to SoleilMCP — never replaced by a service-account credential. This invariant ensures that SoleilMCP's per-tool permission check is performed against the actual end user's identity, not against SoleilAI's service identity. Similarly, SoleilMCP forwards the user's UPN as a bound query parameter to SoleilDW rather than substituting its consumer identity. The result is a complete identity chain from the browser to the data warehouse row, auditable at every step.

8.3 Pre-Filtering as a Security Invariant

Both the data access path and the RAG retrieval path enforce a "pre-filter only" rule: access-scope filtering is applied inside the database query before any results are scored or returned, not as a post-hoc operation on an unfiltered result set. For data access, this means UserStudentAccessResolved is joined within the named query SQL template — not appended by the caller. For RAG, this means the security class WHERE predicate is evaluated by pgvector before cosine-similarity ranking.

This design eliminates a class of vulnerabilities that arise when access controls are applied after query execution: even if a post-filter were incorrectly omitted, no unauthorized data would be returned by the pre-filtered query. The invariant is enforced architecturally — the named query SQL is stored server-side and never exposed or modifiable by consumers — rather than by code review alone.

8.4 Privacy-by-Design Data Handling

Several design decisions reflect privacy-by-design principles:

- Tool result content is never persisted to the conversation database. Student data exists onlyin-memory for the duration of a single chat turn.
- User query text is never stored in audit logs or observability sinks; only character counts arerecorded.
- Bearer tokens are never logged at any log level.
- RAG query strings are not stored; only query_length is recorded in RAGQueryLog.
- Downloaded tool results use UUID-prefixed filenames with no user PII.
- The vLLM server is not reachable from outside the Docker internal network.

8.5 Prohibited Query Patterns

SoleilMCP enforces a restricted prompt policy that prohibits certain categories of queries regardless of the requester's role. No ToolPermission grant may be issued for named queries that would enable:

- Teacher performance ranking or comparison (labour-relations and legal exposure).
- Bulk unfiltered export of student PII (FERPA data minimization violation).
- Cross-school queries by principals (exceeds access scope; RLS handles automatically).

When SoleilDW's row-level security returns an empty result set because the caller lacks access to the requested scope, SoleilMCP returns the empty result without error, warning, or disclosure of the existence of out-of-scope data. The LLM responds by indicating that no matching data is accessible within the user's scope.

9. Implementation Considerations

9.1 Technology Stack

The Soleil Suite is built on a deliberately conservative technology stack — established frameworks with long support horizons, minimal dependencies, and well-understood security properties:

Component	Language	Framework	Database	Key Libraries
SoleilESB	Python 3.x	Django + Celery	PostgreSQL	pika, celery, django-celery-beat
SoleilDW	Python 3.12+	Django 5.x	PostgreSQL 15+	pika (aio-pika), apscheduler, msal, pgvector
SoleilMCP	Python 3.12+	Django 5.x + FastAPI	PostgreSQL 16+ (pg	vector)mcp SDK, httpx, msal, odfplumber, python-docx
SoleilAI	Python 3.12+	FastAPI + Uvicorn	SQLite	mcp SDK, httpx, python-jose, aiosqlite, MSAL.js
LLM Backend	—	vLLM	—	OpenAI-compatible /v1/chat/completions API
Shared infra	—	nginx, RabbitMQ, Docker	—	Docker Compose, shared AMQP broker

Table 8: Technology stack across the Soleil Suite.

9.2 The Model Context Protocol

The use of the Model Context Protocol (MCP) as the integration standard between SoleilAI and SoleilMCP is a significant architectural choice. MCP provides a vendor-neutral, open-standard mechanism for LLM tool integration — the same protocol used by Claude, and increasingly supported across the AI ecosystem. This means that SoleilMCP could, in principle, serve not only SoleilAI but any MCP-compatible AI client, enabling future integrations (e.g., desktop AI assistants, IDE plugins) without changes to the data access or security layers.

Equally important, SoleilAI contains no hard-coded tool names, domain logic, or data schemas. The tool list is fetched from SoleilMCP at runtime. Adding a new named query to SoleilDW, syncing the catalog in SoleilMCP, and assigning ToolPermission grants makes the new tool immediately available to the LLM — the only deployment artifact required is a configuration change, not a code change in any component.

9.3 On-Premises LLM Considerations

The vLLM server provides an OpenAI-compatible inference API that makes SoleilAI model-agnostic. Any instruction-tuned model served by vLLM can be substituted without changing application code. In practice, the choice of model involves trade-offs between instruction-following capability (particularly for multi-step tool-call chains), context window size, inference latency, and GPU memory requirements.

The Soleil architecture accommodates models that emit tool calls in either OpenAI format (tool_calls deltas) or Mistral format ([TOOL_CALLS] JSON blocks), with automatic detection and normalization in the Tool Orchestrator. Control-token sanitization handles models that leak chat-template tokens into their output stream — a common artifact of models fine-tuned with aggressive chat templates.

Context-window management is handled by dropping oldest non-system messages when the estimated token count approaches the configured budget. The system message — which contains the tool-use policy and RAG-injected context — is never dropped, ensuring that the LLM always operates within its intended behavioral constraints.

9.4 Operational Monitoring

Each component exposes operational visibility appropriate to its role: SoleilESB provides Celery Flower for task monitoring and a connector Interface Monitoring view. SoleilDW provides a job execution history for all scheduled tasks and a DataRequestLog viewer. SoleilMCP provides an audit log viewer with filtering by date, caller, tool, and status, and a dashboard summarizing tool call volumes. SoleilAI provides a /api/status endpoint reporting the health of the vLLM server, MCP connection, and RAG availability.

9.5 Graceful Degradation

The system is designed to degrade gracefully when components are unavailable. If SoleilMCP is unreachable at startup, SoleilAI starts with MCP and RAG disabled — users can still converse with the LLM from its training knowledge. If the vLLM server is unreachable, the chat endpoint returns a structured SSE error event rather than an HTTP 5xx response. If the RAG endpoint times out (2-second limit), the LLM call proceeds without document context. If a tool call fails, the LLM is instructed to answer from its own knowledge without retrying — preventing compounding failures from cascading into user-visible errors.

10. Related Work

The concept of retrieval-augmented generation for grounding LLM responses in external knowledge was introduced by Lewis et al. (2020) and has since been widely adopted. The Soleil architecture applies this pattern within a strict security envelope, extending it with multi-class document security filtering that the original formulation did not address.

The Model Context Protocol (MCP), released by Anthropic in 2024, provides the open standard that enables SoleilAI and SoleilMCP to interoperate without vendor lock-in. MCP's design — tools expose JSON Schema-typed parameters and return structured content items — maps naturally to the named query catalog pattern used by SoleilDW, enabling the 1:1 mapping between named queries and MCP tools that eliminates the need for manual tool registration code.

Approaches to privacy-preserving AI in healthcare (e.g., federated learning, differential privacy) share the motivation of keeping sensitive data under institutional control. The Soleil approach differs in that it does not require modifying the LLM training process; instead, it constrains what data the LLM can access at inference time through architectural boundaries rather than mathematical privacy guarantees. This is appropriate for an interactive query assistant where the goal is correct access control rather than statistical privacy.

Educational data warehousing research (Romero & Ventura, 2010; Baker, 2010) has established the value of consolidating heterogeneous educational data into integrated analytical stores. The Soleil architecture extends this tradition by making the warehouse queryable in natural language while preserving the access controls that educational privacy law requires.

11. Conclusions

This paper has presented a reference architecture for sovereign AI in school Divisions, instantiated as the Soleil Suite at Sunrise School Division. The architecture demonstrates that K–12 educational institutions can deliver sophisticated natural-language AI assistance — grounded in authoritative, role-scoped student data and institutional documents — without transmitting student records to external AI providers and without introducing a single access-control bypass.

The key contributions of this architecture are:

1. **Security as a data domain.** By modelling identity, role, and access scope as first-class warehouse data (DimUser, access-grant tables, UserStudentAccessResolved) and applying row-level filtering as a mandatory, pre-computed WHERE predicate inside named queries, the architecture makes unauthorized data access architecturally difficult rather than merely policy-prohibited.
2. **Broker-mediated data access.** By routing all external data requests through RabbitMQ DataRequest messages rather than exposing database ports or HTTP APIs, SoleilDW ensures that no consumer — including the AI system — can construct arbitrary queries or access the warehouse outside of pre-approved, parameterized named queries.
3. **End-to-end identity propagation.** The architecture maintains a complete, unbroken identity chain from the browser to the data warehouse row, with the authenticated user's Bearer token forwarded verbatim through the chain and their UPN injected as a bound parameter on data requests. No service-account substitution occurs at any point.
4. **Zero-code tool extensibility.** The combination of SoleilDW's named query catalog, SoleilMCP's catalog sync, and SoleilAI's dynamic tool list means that new data capabilities can be added — a new attendance report, a new roster query — through configuration changes alone, without modifying application code in any component.
5. **Pre-filtered RAG with security classification.** The RAG document corpus is filtered by security class inside the vector search query, ensuring that restricted document content is never scored against an unauthorized user's query. The security classification model maps naturally onto the educational role hierarchy (teacher, principal, superintendent, administrative staff, public).

The architecture is not without trade-offs. The broker-mediated data access pattern adds latency compared to a direct database query — a round-trip through RabbitMQ introduces tens to hundreds of milliseconds over a direct SQL connection. For interactive AI use, this is acceptable given the LLM inference latency that dominates total response time. The requirement to maintain UserStudentAccessResolved as a precomputed table introduces a refresh lag: a teacher newly assigned a section will not see their new students in AI query results until the AccessResolutionETL job runs — a delay that must be managed operationally.

For school Divisions evaluating sovereign AI deployments, the Soleil architecture offers a documented, production-tested reference. The component separation is clean enough that institutions with existing data warehouse or ETL infrastructure can adopt individual components: a Division with an existing PowerSchool data warehouse might deploy only SoleilMCP and SoleilAI against their existing named query infrastructure. The use of open standards (MCP, OpenAI-compatible vLLM API, AMQP, pgvector, Entra ID) ensures that the architecture is not tied to any single vendor's ecosystem.

As AI capabilities advance and institutional comfort with AI-assisted administration grows, the architecture provides a principled foundation for expanding the system's scope — additional data domains, agentic workflows, multi-modal input — while preserving the privacy and governance invariants that educational law and professional ethics require.

References

Anthropic. (2024). *Model Context Protocol specification*. https://modelcontextprotocol.io

Baker, R. S. J. d. (2010). Data mining for education. *International Encyclopedia of Education*, 7(3), 112–118.

Family Educational Rights and Privacy Act (FERPA), 20 U.S.C. § 1232g (1974).

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

Personal Information Protection and Electronic Documents Act (PIPEDA), S.C. 2000, c. 5 (Canada).

Romero, C., & Ventura, S. (2010). Educational data mining: A review of the state of the art. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 40(6), 601–618.

Tulloch, G. (2026). *Soleil Suite — Project documentation: SoleilAI SRS, SoleilAI PSD, SoleilMCP SRS, SoleilDW SRS, SoleilDW SAD*. Sunrise School Division (internal).

vLLM Project. (2023). *vLLM: Easy, fast, and cheap LLM serving for everyone*. https://docs.vllm.ai

Web Content Accessibility Guidelines (WCAG) 2.1. (2018). W3C Recommendation. https://www.w3.org/TR/WCAG21/